Theory of Topics Used in Boston Housing Price Prediction using Deep Neural Network

This practical implements a **Linear Regression problem using a Deep Neural Network (DNN)** to predict house prices from the Boston Housing dataset.

---

1. What is Linear Regression?

Linear Regression is a **supervised machine learning algorithm** used for predicting continuous numerical values.

In this problem:

- **Input Features (X)** → crime rate, number of rooms, tax, age, etc.
- **Output (Y)** → house price (medv)

The relation is approximately:

y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b

Where:

- (x_1, x_2, ... x_n) = input features
- (w_1, w_2, ... w_n) = weights learned by model
- (b) = bias
- (y) = predicted house price

The model tries to find the best weights to minimize prediction error.

---

2. What is Deep Neural Network (DNN)?

A Deep Neural Network is an Artificial Neural Network having:

- Input layer
- One or more hidden layers
- Output layer

In your code:

```
model = Sequential()

model.add(Dense(64, activation='relu', input_shape=(13,)))

model.add(Dense(32, activation='relu'))

model.add(Dense(1))
```

Architecture:

Layer	Neurons	Purpose
Input Layer	13	Accepts 13 features
Hidden Layer 1	64	Learns patterns
Hidden Layer 2	32	Learns deeper relationships
Output Layer	1	Predicts house price

---

3. Boston Housing Dataset

The Boston Housing dataset contains housing information of Boston city.

Features Used

Feature	Meaning
crim	Crime rate
zn	Residential land proportion
indus	Industrial area proportion
chas	Charles river dummy variable
nox	Nitric oxide concentration
rm	Average number of rooms
age	Age of houses
dis	Distance to employment centers
rad	Highway accessibility
tax	Property tax rate
ptratio	Student-teacher ratio
b	Proportion of Black population
lstat	Lower status population percentage
medv	Median house price (target)

---

4. Importing Libraries

```
import numpy as np
import pandas as pd
```

NumPy

Used for numerical computations and arrays.

Pandas

Used for:

- reading CSV files
- handling tables/dataframes

---

5. Feature and Target Separation

```
x = df.iloc[:,:-1]
y = df.iloc[:,-1]
```

- x contains all input features
- y contains target column (medv)

---

6. Train-Test Split

```
x_train,x_test,y_train,y_test = train_test_split(
    x,y,test_size=0.2,random_state=40)
```

Dataset is divided into:

Part	Percentage
Training Data	80%
Testing Data	20%

Purpose

- Training set → used to train model
- Testing set → used to evaluate model performance

---

7. Feature Scaling (Standardization)

```
scalar = StandardScaler()
x_train = scalar.fit_transform(x_train)
x_test = scalar.transform(x_test)
```

Neural networks work better when features are on similar scale.

StandardScaler transforms data using:

genui{"math_block_widget_always_prefetch_v2":{"content":"z = \frac{x - \mu}{\sigma}"}}

Where:

- (x) = original value
- (\mu) = mean
- (\sigma) = standard deviation

Result:

- mean becomes 0
- standard deviation becomes 1

---

8. TensorFlow and Keras

import tensorflow as tf

TensorFlow

Google's deep learning framework.

Keras

High-level API used for building neural networks easily.

---

9. Sequential Model

model = Sequential()

Sequential model means layers are added one after another sequentially.

---

10. Dense Layer

Dense(64, activation='relu')

Dense layer means every neuron is connected to every neuron of next layer.

---

11. Activation Function (ReLU)

activation='relu'

ReLU stands for Rectified Linear Unit.

Formula:

f(x)=\max(0,x)

Meaning:

- if input > 0 → output same

- if input < 0 → output 0

Why ReLU?

- Fast training
- Avoids vanishing gradient problem
- Most commonly used activation function

---

12. Output Layer

model.add(Dense(1))

Only one neuron because:

- output is a single continuous value (house price)

No activation function used because regression problems require real numeric outputs.

---

13. Model Compilation

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

Compilation defines:

- optimizer
- loss function
- evaluation metrics

---

14. Optimizer (Adam)

Adam optimizer updates weights automatically during training.

Purpose:

- minimize loss
- improve predictions

Advantages:

- Fast
- Efficient

- Adaptive learning rate

---

15. Loss Function (MSE)

loss='mse'

MSE = Mean Squared Error

Formula:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$$

Where:

- y_i = actual value
- $\hat{y}_i$ = predicted value

Lower MSE means better predictions.

---

16. MAE (Mean Absolute Error)

metrics=['mae']

Formula:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i-\hat{y}_i|$$

Measures average absolute difference between actual and predicted values.

Lower MAE means better accuracy.

---

17. Model Training

```
model.fit(
    x_train,
    y_train,
    epochs=100,
    batch_size=16
)
```

Epoch

One complete pass of entire training data through neural network.

Here:

- model trained for 100 iterations.

Batch Size

Number of samples processed before updating weights.

Batch size = 16 means:

- 16 rows processed at a time.

---

18. Forward Propagation

During training:

1. Input goes through layers
2. Predictions are generated
3. Loss is calculated

This process is called Forward Propagation.

---

19. Backpropagation

Neural network adjusts weights using errors.

Steps:

1. Calculate loss
2. Find gradients
3. Update weights using optimizer

This is called Backpropagation.

---

20. Prediction

y_pred = model.predict(x_test)

The trained model predicts house prices for unseen test data.

---

21. Model Evaluation

mse = mean_squared_error(y_test, y_pred)

r2square = r2_score(y_test,y_pred)

---

22. R^2 Score (Coefficient of Determination)

Measures how well model explains variance in data.

Formula:

R^2 = 1 - \frac{\sum (y_i-\hat{y}_i)^2}{\sum (y_i-\bar{y})^2}

Range:

- 1 → perfect prediction
- 0 → poor prediction

Higher R^2 is better.

---

23. Interpretation of Your Training Output

From your epochs:

- Initial loss was very high (~543)
- Final loss reduced to around 5–7
- MAE reduced from ~21 to ~1.7

This means:

- model learned successfully
- prediction error decreased significantly
- neural network fits data well

---

24. Warning Message Explanation

UserWarning: Do not pass an input_shape argument to a layer

This warning occurs because newer Keras versions prefer:

```
from tensorflow.keras import Input

model = Sequential([
    Input(shape=(13,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])
```

Your code still works correctly.

---

25. Complete Workflow of the Practical

Load Dataset

↓

Separate Features & Target

↓

Split Train/Test Data

↓

Scale Features

↓

Build Neural Network

↓

Compile Model

↓

Train Model

↓

Predict Values

↓

Evaluate Performance

---

26. Why DNN Instead of Traditional Linear Regression?

Traditional Linear Regression:

- learns only linear relationships

Deep Neural Network:

- learns complex nonlinear patterns
- better accuracy
- handles feature interactions efficiently

---

27. Advantages of This Approach

- Better prediction capability
- Handles complex data patterns
- Automatic feature learning
- Good performance on regression tasks

---

28. Conclusion

In this practical:

- Boston Housing dataset was used
- Features were standardized
- Deep Neural Network was built using TensorFlow/Keras
- Model trained using Adam optimizer
- MSE and R^2 used for evaluation
- Neural network successfully predicted house prices with low error

This practical demonstrates how Deep Learning can be used for Regression problems effectively.

13. Model Compilation — Detailed Explanation

After building the neural network architecture, the next step is:

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

Compilation is the process where we configure:

1. **Optimizer** → How model learns
2. **Loss Function** → How error is measured
3. **Metrics** → How performance is displayed

Without compilation, the neural network cannot start training.

Why Compilation is Needed?

A neural network initially has random weights.

Example:

Predicted Price = 12

Actual Price = 24

The model must:

1. Measure error
2. Understand how wrong it is
3. Update weights

4. Improve predictions

compile() defines the rules for this learning process.

Structure of compile()

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

Parameter	Purpose
optimizer	Updates weights
loss	Calculates error
metrics	Shows performance during training

A. Loss Function (loss='mse')

Loss function tells:

"How bad are the predictions?"

The model tries to minimize this loss.

Mean Squared Error (MSE)

You used:

loss='mse'

Formula:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$$

Where:

Symbol	Meaning
y_i	Actual value
$\hat{y}_i$	Predicted value
n	Number of samples

Step-by-Step Example of MSE

Suppose:

Actual Price	Predicted Price
20	18
25	30
15	10

Step 1: Error

$$
y_i - \hat{y}_i
$$

Actual	Predicted	Error
20	18	2
25	30	-5
15	10	5

Step 2: Square Errors

$$
(error)^2
$$

Error	Squared Error
2	4
-5	25
5	25

Step 3: Average

$$
MSE = \frac{4+25+25}{3}
$$

$$
MSE = 18
$$

Why Squaring is Used?

Squaring gives:

- Positive values only
- Higher penalty to large errors

Example:

- Error = 10 → squared = 100
- Error = 2 → squared = 4

Large mistakes become heavily punished.

Why MSE is Good for Regression?

Regression predicts continuous values like:

- house price
- salary
- temperature

MSE works well because:

- differentiable
- smooth optimization
- strongly penalizes large errors

B. Metrics (metrics=['mae'])

Metrics are used for monitoring performance during training.

They do not directly update weights.

Mean Absolute Error (MAE)

Formula:

MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i-\hat{y}_i|

Difference from MSE:

- Uses absolute value
- No squaring

Example of MAE

Actual	Predicted	Error
20	18	2
25	30	5
15	10	5

$$
MAE = \frac{2+5+5}{3}
$$

$$
MAE = 4
$$

Meaning:

On average, prediction is off by 4 units.

Difference Between MSE and MAE

MSE	MAE
Squares errors	Uses absolute errors
Penalizes large errors heavily	Treats all errors equally
Used for optimization	Easier interpretation
Sensitive to outliers	More robust

Why Both are Used Together?

- MSE helps training better
- MAE gives understandable error

Example:

MAE = 2

Means:

Predictions are off by approximately 2 units on average.

---

14. Optimizer (Adam) — Detailed Explanation

You used:

optimizer='adam'

The optimizer is the **brain of learning**.

Its job:

Update weights and biases to reduce loss.

What Does Optimizer Do?

Initially weights are random.

Example:

Weight = 0.5

Prediction = Wrong

Optimizer changes weights:

0.5 → 0.7 → 0.9 → ...

until predictions improve.

Neural Network Learning Process

Input Data

↓

Prediction

↓

Loss Calculation

↓

Optimizer Updates Weights

↓

Better Prediction

This repeats for every epoch.

Gradient Descent Concept

Optimizers work using Gradient Descent.

Goal:

Reach minimum loss.

Imagine standing on a mountain:

- top = high loss

- bottom = minimum loss

Optimizer moves downhill step-by-step.

Weight Update Formula

Basic gradient descent:

$$w_{new}=w_{old}-\eta \frac{\partial L}{\partial w}$$

Where:

Symbol	Meaning
(w)	Weight
(η)	Learning rate
(L)	Loss
$(\frac{\partial L}{\partial w})$	Gradient

Problem with Basic Gradient Descent

Traditional gradient descent can:

- be slow
- oscillate
- get stuck
- require manual learning rate tuning

Adam Optimizer

Adam stands for:

Adaptive Moment Estimation

It is one of the most popular optimizers in deep learning.

Adam combines:

1. Momentum
2. RMSProp

to improve training speed and stability.

How Adam Works

Adam:

- remembers past gradients
- adjusts learning rate automatically
- updates weights efficiently

This makes training:

- faster
- smoother
- more accurate

Advantages of Adam

Advantage	Explanation
Fast training	Converges quickly
Adaptive learning rate	Different learning rate for each weight
Stable	Less oscillation
Good default choice	Works well in most problems

Why Adam is Used in Your Project

Boston Housing dataset:

- has multiple features
- nonlinear relationships
- scaled data

Adam handles these efficiently and trains faster than simple gradient descent.

Training Flow with Adam in Your Code

1. Input data given
2. Model predicts house price
3. MSE loss calculated
4. Backpropagation computes gradients
5. Adam updates weights
6. Loss decreases

7. Predictions improve

Understanding Your Epoch Output

Example:

Epoch 1:

loss: 543

Epoch 100:

loss: 5

This means:

- Adam successfully updated weights
- Model learned patterns
- Prediction error reduced drastically

Simple Analogy

Imagine learning basketball shots.

- Loss function = tells how far ball is from basket
- Optimizer = tells how to adjust aim
- Epoch = one full practice session

After many practice sessions:

- aim improves
- errors reduce

Same happens in neural networks.